Date: 15/10/2026

C01 - ASSESSMENT TOOL 2 - CODING CHALLENGE

Course Code: DSA03

Course Title: Natural Language Processing

Reg.no: 192425005 **Name:** S. Yuva Guru Mani Varma

Q1. Intelligent Email and Password Validator Using Regular Expressions

Aim: To develop a Python program that validates email addresses, passwords, and mobile numbers using regular expressions.

Python program :

```
import re

def validate_email(email):
    pattern = r'^[A-Za-z][A-Za-z0-9._]*@[A-Za-z]+\.(com|org|edu|net|in)$'
    return re.fullmatch(pattern, email) is not None

def validate_password(password):
    pattern = r'^(?=.*[A-Z])(?=.*[a-z])(?=.*\d)(?=.*[@#$%&!]).{8,}$'
    return re.fullmatch(pattern, password) is not None

def validate_mobile(mobile):
    pattern = r'^[6-9]\d{9}$'
    return re.fullmatch(pattern, mobile) is not None

email = input("Enter email address: ")
password = input("Enter password: ")
mobile = input("Enter mobile number: ")

print("\n--- Validation Results ---")

if validate_email(email):
    print("Valid Email")
else:
    print("Invalid Email")

if validate_password(password):
    print("Strong Password")
else:
    print("Weak Password")

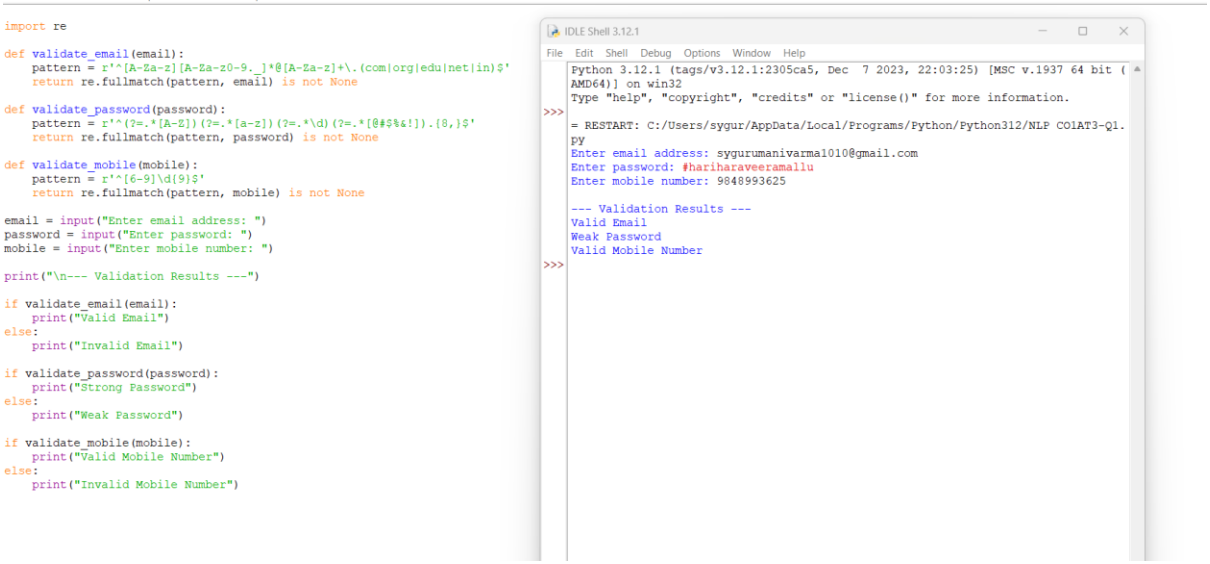
if validate_mobile(mobile):
```

```
    print("Valid Mobile Number")
else:
    print("Invalid Mobile Number")
```

Output:

Enter email address: sygurumanivarma1010@gmail.com
Enter password: #hariharaveeramallu
Enter mobile number: 9848993625

--- Validation Results ---
Valid Email
Weak Password
Valid Mobile Number



```
import re

def validate_email(email):
    pattern = r'^[A-Za-z][A-Za-z0-9._]*@[A-Za-z]+\.(com|org|edu|net|in)$'
    return re.fullmatch(pattern, email) is not None

def validate_password(password):
    pattern = r'^(?=.*[A-Z])(?=.*[a-z])(?=.*\d)(?=.*[@#$%&!]).{8,}$'
    return re.fullmatch(pattern, password) is not None

def validate_mobile(mobile):
    pattern = r'^[6-9]\d{9}$'
    return re.fullmatch(pattern, mobile) is not None

email = input("Enter email address: ")
password = input("Enter password: ")
mobile = input("Enter mobile number: ")

print("\n--- Validation Results ---")

if validate_email(email):
    print("Valid Email")
else:
    print("Invalid Email")

if validate_password(password):
    print("Strong Password")
else:
    print("Weak Password")

if validate_mobile(mobile):
    print("Valid Mobile Number")
else:
    print("Invalid Mobile Number")
```

```
IDLE Shell 3.12.1
File  Edit  Shell  Debug  Options  Window  Help
Python 3.12.1 (tags/v3.12.1:2305ca5, Dec  7 2023, 22:03:25) [MSC v.1937 64 bit (
AMD64)] on win32
Type "help", "copyright", "credits" or "license()" for more information.
>>>
= RESTART: C:/Users/sygur/AppData/Local/Programs/Python/Python312/NLP C01AT3-Q1.
py
Enter email address: sygurumanivarma1010@gmail.com
Enter password: #hariharaveeramallu
Enter mobile number: 9848993625

--- Validation Results ---
Valid Email
Weak Password
Valid Mobile Number
>>>
```

Q2. Design a Finite State Automata (FSA) Simulator

Aim: To implement a Deterministic Finite Automaton (DFA) simulator that accepts or rejects strings according to the specified language.
The following program implements the example in your question paper: strings ending with ab.

DFA transition table

Current state	Input a	Input b
q0	q1	q0
q1	q1	q2
q2	q1	q0

- Initial state: q0
- Final state: q2
- Input alphabet: {a, b}

Python program :

```
def simulate_dfa(string):
    states = ["q0", "q1", "q2"]
    alphabet = ["a", "b"]

    transition = {
        "q0": {"a": "q1", "b": "q0"},
        "q1": {"a": "q1", "b": "q2"},
        "q2": {"a": "q1", "b": "q0"}
    }

    initial_state = "q0"
    final_states = {"q2"}

    current_state = initial_state
    path = [current_state]

    for symbol in string:
        if symbol not in alphabet:
            print("Invalid input symbol:", symbol)
            return

        current_state = transition[current_state][symbol]
        path.append(current_state)

    print("Input String:", string)
    print("Transition Path:", " -> ".join(path))

    if current_state in final_states:
        print("Accepted")
    else:
        print("Rejected")


n = int(input("Enter number of strings: "))

for i in range(n):
    string = input("Enter string: ")
    simulate_dfa(string)
    print()
```

Output :

Enter number of strings: 5
Enter string: abaab
Input String: abaab
Transition Path: q0 -> q1 -> q2 -> q1 -> q1 -> q2
Accepted

Enter string: asdfg
Invalid input symbol: s

```python
def simulate_dfa(string):
    states = ["q0", "q1", "q2"]
    alphabet = ["a", "b"]

    transition = {
        "q0": {"a": "q1", "b": "q0"},
        "q1": {"a": "q1", "b": "q2"},
        "q2": {"a": "q1", "b": "q0"}
    }

    initial_state = "q0"
    final_states = {"q2"}

    current_state = initial_state
    path = [current_state]

    for symbol in string:
        if symbol not in alphabet:
            print("Invalid input symbol:", symbol)
            return

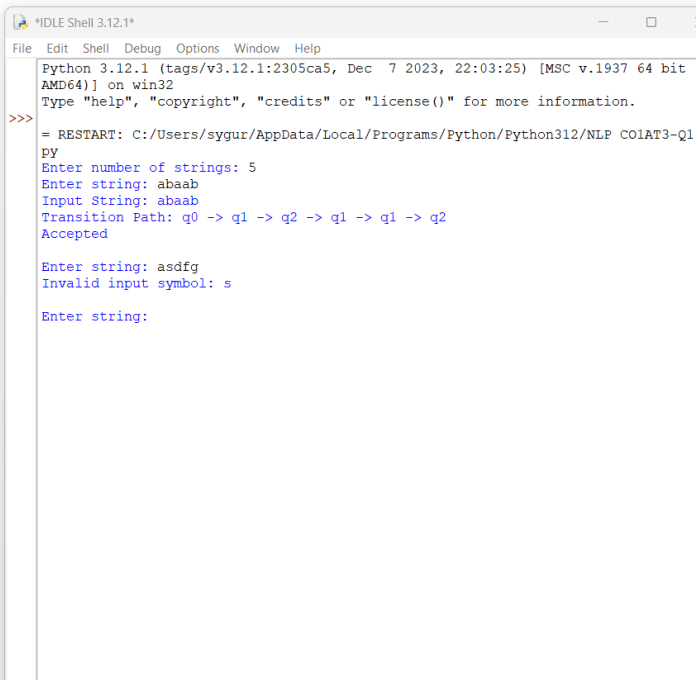
        current_state = transition[current_state][symbol]
        path.append(current_state)

    print("Input String:", string)
    print("Transition Path:", " -> ".join(path))

    if current_state in final_states:
        print("Accepted")
    else:
        print("Rejected")


n = int(input("Enter number of strings: "))

for i in range(n):
    string = input("Enter string: ")
    simulate_dfa(string)
    print()
```



3. Smart Pattern Matching Engine Using Regular Expressions

Aim: To develop a Python text-search engine that supports word, prefix, suffix, date, phone number, hashtag, and mention searches using regular expressions.

Python program :

```
import re

text = """Meeting on 12/09/2026
Call 9876543210
#NLP
@OpenAI
natural language processing"""

def search_pattern(pattern, description):
    matches = re.findall(pattern, text, re.IGNORECASE)

    print("\n" + description + ":")
    if matches:
        for match in matches:
            print(match)
    else:
        print("No matches found.")

while True:
    print("\n--- Smart Pattern Matching Engine ---")
    print("1. Search Date")
    print("2. Search Phone Number")
    print("3. Search Hashtag")
```

```
print("4. Search Mention")
print("5. Search Prefix")
print("6. Search Suffix")
print("7. Search Word")
print("8. Exit")

choice = input("Enter your choice: ")

if choice == "1":
    search_pattern(
        r'\b\d{2}/\d{2}/\d{4}\b', "Dates"
    )

elif choice == "2":
    search_pattern(
        r'\b[6-9]\d{9}\b', "Phone Numbers"
    )

elif choice == "3":
    search_pattern(
        r'(?<!\w)#[A-Za-z0-9_]+', "Hashtags"
    )

elif choice == "4":
    search_pattern(
        r'(?<!\w)@[A-Za-z0-9_]+', "Mentions"
    )

elif choice == "5":
    prefix = input("Enter prefix: ")
    pattern = r'\b' + re.escape(prefix) + r'\w*'
    search_pattern(pattern, "Prefix Matches")

elif choice == "6":
    suffix = input("Enter suffix: ")
    pattern = r'\b\w*' + re.escape(suffix) + r'\b'
    search_pattern(pattern, "Suffix Matches")

elif choice == "7":
    word = input("Enter word: ")
    pattern = r'\b' + re.escape(word) + r'\b'
    search_pattern(pattern, "Word Matches")

elif choice == "8":
    print("Exiting program.")
    break

else:
    print("Invalid choice. Try again.")
```

Output:

```
-- Smart Pattern Matching Engine ---
1. Search Date
2. Search Phone Number
3. Search Hashtag
4. Search Mention
5. Search Prefix
6. Search Suffix
7. Search Word
8. Exit
Enter your choice: 2

Phone Numbers:
9876543210

--- Smart Pattern Matching Engine ---
1. Search Date
2. Search Phone Number
3. Search Hashtag
4. Search Mention
5. Search Prefix
6. Search Suffix
7. Search Word
8. Exit
Enter your choice: 1

Dates:
12/09/2026
```

```python
import re

text = """Meeting on 12/09/2026
Call 9876543210
#NLP
@OpenAI
natural language processing"""

def search_pattern(pattern, description):
    matches = re.findall(pattern, text, re.IGNORECASE)

    print("\n" + description + ":")
    if matches:
        for match in matches:
            print(match)
    else:
        print("No matches found.")

while True:
    print("\n--- Smart Pattern Matching Engine ---")
    print("1. Search Date")
    print("2. Search Phone Number")
    print("3. Search Hashtag")
    print("4. Search Mention")
    print("5. Search Prefix")
    print("6. Search Suffix")
    print("7. Search Word")
    print("8. Exit")

    choice = input("Enter your choice: ")

    if choice == "1":
        search_pattern(
            r'\b\d{2}/\d{2}/\d{4}\b', "Dates"
        )

    elif choice == "2":
        search_pattern(
            r'\b[6-9]\d{9}\b', "Phone Numbers"
        )

    elif choice == "3":
        search_pattern(
            r'(?<!\w)#[A-Za-z0-9_]+', "Hashtags"
        )

    elif choice == "4":
        search_pattern(
            r'(?<!\w)@[A-Za-z0-9_]+', "Mentions"
        )

    elif choice == "5":
```

IDLE Shell 3.12.1

File Edit Shell Debug Options Window Help

```
py

--- Smart Pattern Matching Engine ---
1. Search Date
2. Search Phone Number
3. Search Hashtag
4. Search Mention
5. Search Prefix
6. Search Suffix
7. Search Word
8. Exit
Enter your choice: 2

Phone Numbers:
9876543210

--- Smart Pattern Matching Engine ---
1. Search Date
2. Search Phone Number
3. Search Hashtag
4. Search Mention
5. Search Prefix
6. Search Suffix
7. Search Word
8. Exit
Enter your choice: 1

Dates:
12/09/2026

--- Smart Pattern Matching Engine ---
1. Search Date
2. Search Phone Number
3. Search Hashtag
4. Search Mention
5. Search Prefix
6. Search Suffix
7. Search Word
8. Exit
Enter your choice:
```